svanberg.cpp 与 athinput.figure2 逐段说明

一、src/svanberg.cpp

下面按原文件顺序收录代码，左侧数字是源文件行号。说明只针对这个 C++ 文件及后面的输入文件。当前输入采用球坐标、绝热能量方程和轴对称网格；代码中保留的柱坐标与其他边界分支会分别注明是否执行。

读数组时先区分：`u` 是守恒变量，保存密度、动量密度和总能量密度；`w`、`prim` 是原始变量，保存密度、速度和压强。同样的下标数字在两类数组中不代表同一种物理量。

1. 文件注释与头文件（第 1–36 行）

```
1  //========================================================================================
2  // Athena++ astrophysical MHD code
3  // Copyright(C) 2014 James M. Stone <jmstone@princeton.edu> and other code contributors
4  // Licensed under the 3-clause BSD License, see LICENSE file for details
5  //========================================================================================
6  //! \file disk.cpp
7  //! \brief Initializes stratified Keplerian accretion disk in both cylindrical and
8  //! spherical polar coordinates.  Initial conditions are in vertical hydrostatic eqm.
9
10 // C headers
11
12 // C++ headers
13 #include <algorithm>  // min
14 #include <cstdint>
15 #include <cmath>      // sqrt
16 #include <cstdlib>    // srand
17 #include <cstring>    // strcmp()
18 #include <fstream>
19 #include <iostream>   // endl
20 #include <limits>
21 #include <sstream>    // stringstream
22 #include <stdexcept>  // runtime_error
23 #include <string>     // c_str()
24
25 // Athena++ headers
26 #include "../athena.hpp"
27 #include "../athena_arrays.hpp"
28 #include "../bvals/bvals.hpp"
29 #include "../coordinates/coordinates.hpp"
30 #include "../eos/eos.hpp"
31 #include "../field/field.hpp"
32 #include "../globals.hpp"
33 #include "../hydro/hydro.hpp"
34 #include "../mesh/mesh.hpp"
35 #include "../orbital_advection/orbital_advection.hpp"
36 #include "../parameter_input.hpp"
```

第 1–8 行保留版权、BSD 许可和功能注释。第 6 行的 `disk.cpp` 是沿用的注释文字，实际文件名是 `svanberg.cpp`；注释不会决定编译的问题名称。第 9–12、24–25 行为空行或分类注释，不执行计算。

第 13–23 行是 C++ 标准库：

行	头文件	本文件中的作用
13	`algorithm`	使用 `std::max` 限制密度下限；旁注 `min` 是遗留注释。
14	`cstdint`	提供 `std::uint64_t`，用于噪声散列。

行	头文件	本文件中的作用
15	`cmath`	平方根、幂、指数、对数、四舍五入及 `expm1`。
16	`cstdlib`	遗留包含项；当前噪声没有调用 `srand`。
17	`cstring`	用 `strcmp` 判断坐标系字符串。
18–19	`fstream`、`iostream`	保留的文件与流接口；当前没有直接用它们输出数据。
20	`limits`	获取浮点数最小正规格化正数，构造默认底限。
21–23	`sstream`、`stdexcept`、`string`	保留的字符串流、异常和字符串接口。

第 26–36 行接入 Athena++ 类型：`athena.hpp` 提供 `Real`、变量下标和宏；`athena_arrays.hpp` 提供数组；`bvals.hpp` 提供边界接口；`coordinates.hpp` 提供单元坐标；`eos.hpp` 提供状态方程接口；`field.hpp` 提供边界签名所需的 `FaceField`；`globals.hpp` 提供全局接口；`hydro.hpp` 访问流体数组；`mesh.hpp` 访问网格类；`orbital_advection.hpp` 访问可选轨道平流；`parameter_input.hpp` 读取输入。保留磁场类型接口不等于启用了磁场。

2. 局部函数、参数与 Noise（第 37–56 行）

```
37
38 namespace {
39 void GetCylCoord(Coordinates *pco,Real &rad,Real &phi,Real &z,int i,int j,int k);
40 Real DenProfileCyl(const Real rad, const Real phi, const Real z);
41 Real PoverR(const Real rad, const Real phi, const Real z);
42 Real VelProfileCyl(const Real rad, const Real phi, const Real z);
43 // problem parameters which are useful to make global to this file
44 Real gm0, r0, rho0, dslope, p0_over_r0, pslope, gamma_gas;
45 Real dfloor;
46 Real Omega0, noise_amp, tau;
47 std::uint64_t seed;
48 // SplitMix64: coordinate-keyed noise independent of block decomposition.
49 Real Noise(Real r, Real theta, int component) {
50   std::uint64_t x=seed ^ (std::uint64_t(std::llround(std::log(r)*1.e12))*0x9e3779b97f4a7c15ULL)
51     ^ std::uint64_t(std::llround(theta*1.e12)) ^ (std::uint64_t(component+1)*0xbf58476d1ce4e5b9ULL);
52   x=(x^(x>>30))*0xbf58476d1ce4e5b9ULL;
53   x=(x^(x>>27))*0x94d049bb133111ebULL; x^=x>>31;
54   return 2.0*Real(x>>11)/9007199254740992.0-1.0;
55 }
56 } // namespace
```

第 38 行的匿名 `namespace` 将这些函数和变量限制在本源文件内部。第 39–42 行提前声明四个辅助函数，供后面的初始化和边界代码调用。

第 44–47 行声明共享参数：

变量	含义
`gm0`, `r0`, `rho0`	GM、参考柱半径 R_0、参考中面密度 ρ_0。
`dslope`, `pslope`	密度和声速平方的径向幂指数，本次为 -2 和 -1。
`p0_over_r0`	参考位置的 $P/\rho = c_0^2 = 0.0025$。
`gamma_gas`, `dfloor`	比热比与密度底限。
`Omega0`	参考系旋转角速度，默认 0；不是定义 P_0 的开普勒频率。
`noise_amp`, `tau`, `seed`	噪声幅度、冷却时间、随机种子。

第 49–55 行 `Noise()` 为同一坐标和速度分量返回相同的伪随机值。第 50 行把 `log(r)` 乘 10^{12} 并取整，编码径向坐标；第 51 行把极角和分量编号混入整数。`^` 是按位异或，`>>` 是右移，`ULL` 表示无符号长整型常量，均不是物理公式中的乘方。

第 52–53 行用异或、右移和固定乘数混合比特。第 54 行取高 53 位并除以 $2^{53} = 9007199254740992$，先映射到 $[0, 1)$，再映射到 $[-1, 1)$。这些乘数用于散列，不是额外的物理参数。按坐标生成噪声，使初始化不依赖网格块的处理顺序。

3. 六个边界函数的声明（第 57–76 行）

```
57
58 // User-defined boundary conditions for disk simulations
59 void DiskInnerX1(MeshBlock *pmb, Coordinates *pco, AthenaArray<Real> &prim,FaceField &b,
60                  Real time, Real dt,
61                  int il, int iu, int jl, int ju, int kl, int ku, int ngh);
62 void DiskOuterX1(MeshBlock *pmb, Coordinates *pco, AthenaArray<Real> &prim,FaceField &b,
63                  Real time, Real dt,
64                  int il, int iu, int jl, int ju, int kl, int ku, int ngh);
65 void DiskInnerX2(MeshBlock *pmb, Coordinates *pco, AthenaArray<Real> &prim,FaceField &b,
66                  Real time, Real dt,
67                  int il, int iu, int jl, int ju, int kl, int ku, int ngh);
68 void DiskOuterX2(MeshBlock *pmb, Coordinates *pco, AthenaArray<Real> &prim,FaceField &b,
69                  Real time, Real dt,
70                  int il, int iu, int jl, int ju, int kl, int ku, int ngh);
71 void DiskInnerX3(MeshBlock *pmb, Coordinates *pco, AthenaArray<Real> &prim,FaceField &b,
72                  Real time, Real dt,
73                  int il, int iu, int jl, int ju, int kl, int ku, int ngh);
74 void DiskOuterX3(MeshBlock *pmb, Coordinates *pco, AthenaArray<Real> &prim,FaceField &b,
75                  Real time, Real dt,
76                  int il, int iu, int jl, int ju, int kl, int ku, int ngh);
```

这里先声明函数，尚未设置任何边界值。`Inner/Outer` 表示内侧/外侧，`X1/X2/X3` 表示第 1/2/3 个坐标方向。球坐标下依次为 r, θ, ϕ。

各参数含义相同：`pmb` 是当前网格块，`pco` 是坐标对象，`prim` 是要填写的原始变量数组；`b` 为接口保留的磁场数据；`time`、`dt` 是边界调用时的时间和步长，本函数没有直接使用它们改变平衡值。`il/iu`、`jl/ju`、`kl/ku` 是相应索引范围，`ngh` 是幽灵层数。当前输入只有前四个边界函数会作为用户边界被调用，X3 使用周期条件。

4. InitUserMeshData：读取输入与注册边界（第 77–132 行）

```
77
78 //========================================================================================
79 //! \fn void Mesh::InitUserMeshData(ParameterInput *pin)
80 //! \brief Function to initialize problem-specific data in mesh class.  Can also be used
81 //! to initialize variables which are global to (and therefore can be passed to) other
82 //! functions in this file.  Called in Mesh constructor.
83 //========================================================================================
84
85 void Mesh::InitUserMeshData(ParameterInput *pin) {
86   // Get parameters for gravitatonal potential of central point mass
87   gm0 = pin->GetOrAddReal("problem","GM",0.0);
88   r0 = pin->GetOrAddReal("problem","r0",1.0);
89
90   // Get parameters for initial density and velocity
91   rho0 = pin->GetReal("problem","rho0");
92   dslope = pin->GetOrAddReal("problem","dslope",0.0);
93
94   // Get parameters of initial pressure and cooling parameters
95   if (NON_BAROTROPIC_EOS) {
96     p0_over_r0 = pin->GetOrAddReal("problem","p0_over_r0",0.0025);
97     pslope = pin->GetOrAddReal("problem","pslope",0.0);
98     gamma_gas = pin->GetReal("hydro","gamma");
99   } else {
100    p0_over_r0=SQR(pin->GetReal("hydro","iso_sound_speed"));
101  }
102  Real float_min = std::numeric_limits<float>::min();
103  dfloor=pin->GetOrAddReal("hydro","dfloor",(1024*(float_min)));
104
105  Omega0 = pin->GetOrAddReal("orbital_advection","Omega0",0.0);
106
107  noise_amp=pin->GetOrAddReal("problem","noise_amp",0.01);
108  tau=pin->GetOrAddReal("problem","tau",6.283185307179586e-10);
109  seed=pin->GetOrAddInteger("problem","seed",202211);
110
111  // enroll user-defined boundary condition
112  if (mesh_bcs[BoundaryFace::inner_x1] == GetBoundaryFlag("user")) {
113    EnrollUserBoundaryFunction(BoundaryFace::inner_x1, DiskInnerX1);
114  }
115  if (mesh_bcs[BoundaryFace::outer_x1] == GetBoundaryFlag("user")) {
116    EnrollUserBoundaryFunction(BoundaryFace::outer_x1, DiskOuterX1);
117  }
118  if (mesh_bcs[BoundaryFace::inner_x2] == GetBoundaryFlag("user")) {
```

```
119     EnrollUserBoundaryFunction(BoundaryFace::inner_x2, DiskInnerX2);
120   }
121   if (mesh_bcs[BoundaryFace::outer_x2] == GetBoundaryFlag("user")) {
122     EnrollUserBoundaryFunction(BoundaryFace::outer_x2, DiskOuterX2);
123   }
124   if (mesh_bcs[BoundaryFace::inner_x3] == GetBoundaryFlag("user")) {
125     EnrollUserBoundaryFunction(BoundaryFace::inner_x3, DiskInnerX3);
126   }
127   if (mesh_bcs[BoundaryFace::outer_x3] == GetBoundaryFlag("user")) {
128     EnrollUserBoundaryFunction(BoundaryFace::outer_x3, DiskOuterX3);
129   }
130
131   return;
132 }
```

该函数在建立网格时执行，负责准备后续初始化、边界与冷却共用的参数。

- 第 87–88 行读取 `problem/GM` 和 `problem/r0`，默认分别为 0 和 1。
- 第 91 行 `GetReal` 要求输入必须提供 `rho0`；第 92 行 `GetOrAddReal` 读取 `dslope`，缺省时补入默认值 0。
- 第 95–98 行在绝热能量方程编译下读取声速平方归一化、温度幂指数以及 `hydro/gamma`。当前执行这一分支。
- 第 99–100 行是保留的等温读取分支。但末尾冷却函数直接操作能量，因此不能仅凭这个分支就认定整个文件支持直接改成等温编译。
- 第 102 行取得单精度最小正规格化正数；第 103 行据此给 `dfloor` 一个默认值。本输入已显式设为 10^{-12}，所以实际使用显式值。
- 第 105 行读取参考系的 `Omega0`，输入未给出，使用默认 0。
- 第 107–109 行读取 `noise_amp=0.01`、`tau=2\pi\times10^{-10}` 和 `seed=202211`。
- 第 112–129 行逐个检查六个面的边界标志。只有等于 `user` 才通过 `EnrollUserBoundaryFunction` 注册对应函数。当前 X1、X2 的四个面会注册，X3 的两个面因设置为 `periodic` 而不注册。
- 第 131 行结束函数，不创建流体初始场；初始场由下一个函数填写。

5. ProblemGenerator：逐单元建立初态（第 133–184 行）

```
133
134 //========================================================================================
135 //! \fn void MeshBlock::ProblemGenerator(ParameterInput *pin)
136 //! \brief Initializes Keplerian accretion disk.
137 //========================================================================================
138
139 void MeshBlock::ProblemGenerator(ParameterInput *pin) {
140   Real rad(0.0), phi(0.0), z(0.0);
141   Real den, vel;
142   Real x1, x2, x3;
143
144   OrbitalVelocityFunc &vK = porb->OrbitalVelocity;
145   //  Initialize density and momenta
146   for (int k=ks; k<=ke; ++k) {
147     x3 = pcoord->x3v(k);
148     for (int j=js; j<=je; ++j) {
149       x2 = pcoord->x2v(j);
150       for (int i=is; i<=ie; ++i) {
151         x1 = pcoord->x1v(i);
152         GetCylCoord(pcoord,rad,phi,z,i,j,k); // convert to cylindrical coordinates
153         // compute initial conditions in cylindrical coordinates
154         den = DenProfileCyl(rad,phi,z);
155         vel = VelProfileCyl(rad,phi,z);
156         if (porb->orbital_advection_defined)
157           vel -= vK(porb, x1, x2, x3);
158         phydro->u(IDN,k,j,i) = den;
159         phydro->u(IM1,k,j,i) = 0.0;
160         if (std::strcmp(COORDINATE_SYSTEM, "cylindrical") == 0) {
161           phydro->u(IM2,k,j,i) = den*vel;
162           phydro->u(IM3,k,j,i) = 0.0;
163         } else if (std::strcmp(COORDINATE_SYSTEM, "spherical_polar") == 0) {
164           phydro->u(IM2,k,j,i) = 0.0;
165           phydro->u(IM3,k,j,i) = den*vel;
166         }
167
168         // Uniform velocity perturbations in [-0.01 cs,+0.01 cs] in each component.
169         Real cs=std::sqrt(PoverR(rad,phi,z));
170         phydro->u(IM1,k,j,i)+=den*noise_amp*cs*Noise(x1,x2,0);
```

```
171         phydro->u(IM2,k,j,i)+=den*noise_amp*cs*Noise(x1,x2,1);
172         phydro->u(IM3,k,j,i)+=den*noise_amp*cs*Noise(x1,x2,2);
173         if (NON_BAROTROPIC_EOS) {
174           Real p_over_r = PoverR(rad,phi,z);
175           phydro->u(IEN,k,j,i) = p_over_r*phydro->u(IDN,k,j,i)/(gamma_gas - 1.0);
176           phydro->u(IEN,k,j,i) += 0.5*(SQR(phydro->u(IM1,k,j,i))+SQR(phydro->u(IM2,k,j,i))
177                                        + SQR(phydro->u(IM3,k,j,i)))/phydro->u(IDN,k,j,i);
178         }
179       }
180     }
181   }
182
183   return;
184 }
```

第 140–142 行声明局部坐标和临时密度、速度。`rad` 表示柱半径 R；球坐标运行中的 `x1` 是球半径 r，两者不同。第 144 行取得可选轨道速度函数的引用。

第 146–151 行遍历活动单元：`i` 对应 X1，`j` 对应 X2，`k` 对应 X3；`x1v/x2v/x3v` 读取体积平均单元中心。当前 `nx3=1`，但仍保留三重循环。

第 152 行转换到柱坐标。第 154–155 行分别计算式 (24) 的平衡密度和式 (25) 对应的旋转线速度。第 156–157 行只在启用轨道平流时减去轨道速度；当前默认未启用。

第 158 行保存密度，第 159 行把径向动量先设为 0。第 160–166 行按坐标系选择旋转分量：柱坐标时方位动量在 `IM2`；球坐标时在 `IM3`。当前执行第 164–165 行，得到 $m_\theta = 0$、$m_\phi = \rho v_\phi$。

第 169 行由 P/ρ 开方取得局部等温声速。第 170–172 行分别向三个动量加上

$$\delta m_a = \rho\,(0.01) c_s\,\xi_a, \qquad -1 \le \xi_a < 1.$$

`den*` 是必要的，因为 `u(IM*)` 保存动量密度，不是速度。参数 0、1、2 区分三个噪声分量。球坐标运行时噪声用 `x1=r`、`x2=theta` 作坐标键。

第 173–177 行设置总能量：先用 $P = \rho c_s^2$ 计算内能 $\epsilon = P/(\gamma - 1)$，再加上三个动量给出的动能 $K = |\mathbf{m}|^2/(2\rho)$。此时动量已经包含噪声，所以能量也包含扰动动能。第 179–184 行结束循环与函数。噪声仅在初始化时加入，不在这里持续驱动。

6. GetCylCoord：把网格坐标转换为 R、phi、z（第 185–201 行）

```
185
186 namespace {
187 //----------------------------------------------------------------------------------------
188 //! transform to cylindrical coordinate
189
190 void GetCylCoord(Coordinates *pco,Real &rad,Real &phi,Real &z,int i,int j,int k) {
191   if (std::strcmp(COORDINATE_SYSTEM, "cylindrical") == 0) {
192     rad=pco->x1v(i);
193     phi=pco->x2v(j);
194     z=pco->x3v(k);
195   } else if (std::strcmp(COORDINATE_SYSTEM, "spherical_polar") == 0) {
196     rad=std::abs(pco->x1v(i)*std::sin(pco->x2v(j)));
197     phi=pco->x3v(k);
198     z=pco->x1v(i)*std::cos(pco->x2v(j));
199   }
200   return;
201 }
```

第 190 行的 `&rad`、`&phi`、`&z` 是引用输出：函数改变这些变量后，调用者能直接读到结果。

第 191–194 行用于柱坐标运行，直接读取 R, ϕ, z。第 195–198 行用于当前球坐标运行：

$$R = |r\sin\theta|, \qquad \phi = x_3, \qquad z = r\cos\theta.$$

`abs` 保证柱半径非负；`cos(theta)` 保留上下半盘的高度符号。这里转换的是坐标，没有转换速度，也没有改变流体数组。

7. DenProfileCyl：式 (24) 的密度（第 202–214 行）

```
202
203 //----------------------------------------------------------------------------------------
204 //! computes density in cylindrical coordinates
205
206 Real DenProfileCyl(const Real rad, const Real phi, const Real z) {
207   Real den;
208   Real p_over_r = p0_over_r0;
209   if (NON_BAROTROPIC_EOS) p_over_r = PoverR(rad, phi, z);
210   Real denmid = rho0*std::pow(rad/r0,dslope);
211   Real dentem = denmid*std::exp(gm0/p_over_r*(1./std::sqrt(SQR(rad)+SQR(z))-1./rad));
212   den = dentem;
213   return std::max(den,dfloor);
214 }
```

该函数逐项实现

$$\rho = \rho_0 (R/R_0)^{-q_D} \exp\left[\frac{GM}{c_s^2(R)}\left(\frac{1}{r}-\frac{1}{R}\right)\right].$$

- 第 207–208 行声明临时密度，并先赋参考声速平方。
- 第 209 行在当前绝热编译下调用 `PoverR()`，换成当前位置的 $c_s^2(R)$。
- 第 210 行是中面密度 $\rho_0(R/R_0)^{-q_D}$，因为 `dslope=-q_D=-2`。
- 第 211 行中 `sqrt(SQR(rad)+SQR(z))` 就是球半径 r；`gm0/p_over_r` 就是 GM/c_s^2；指数内部的差是 $1/r - 1/R$。
- 第 212 行复制临时结果。第 213 行返回密度与 `dfloor` 的较大值；低于数值底限时才改变解析密度。

`phi` 保留在接口中，但这个轴对称密度解不依赖它。

8. PoverR：局部目标声速平方（第 215–223 行）

```
215
216 //----------------------------------------------------------------------------------------
217 //! computes pressure/density in cylindrical coordinates
218
219 Real PoverR(const Real rad, const Real phi, const Real z) {
220   Real poverr;
221   poverr = p0_over_r0*std::pow(rad/r0, pslope);
222   return poverr;
223 }
```

第 220 行声明返回值。第 221 行计算

$$\frac{P}{\rho} = c_s^2(R) = \mathrm{p0_over_r0}(R/r_0)^{\mathrm{pslope}}.$$

本输入给出 `p0_over_r0=0.0025`、`r0=1`、`pslope=-1`，所以返回 $0.0025/R$。第 222 行返回该值。它不是声速；需要声速时再开平方。`phi` 和 `z` 没进入表达式，因此目标温度在给定柱半径上不随高度变化。

9. VelProfileCyl：式 (25) 对应的旋转线速度（第 224–235 行）

```
224
225 //----------------------------------------------------------------------------------------
226 //! computes rotational velocity in cylindrical coordinates
227
228 Real VelProfileCyl(const Real rad, const Real phi, const Real z) {
229   Real p_over_r = PoverR(rad, phi, z);
230   Real vel = (dslope+pslope)*p_over_r/(gm0/rad) + (1.0+pslope)
231              - pslope*rad/std::sqrt(rad*rad+z*z);
232   vel = std::sqrt(gm0/rad)*std::sqrt(vel) - rad*Omega0;
233   return vel;
234 }
235 } // namespace
```

第 229 行读取局部 c_s^2。第 230–231 行把式 (25) 的无量纲括号存入临时变量 `vel`：

$$B = -(q_D + q_T)h^2 + (1 - q_T) + q_T R/r.$$

对应关系为 `dslope=-q_D`、`pslope=-q_T`，以及

$$\frac{\text{p_over_r}}{\text{gm0/rad}} = \frac{c_s^2}{GM/R} = h^2.$$

因此第 230 行的第一项是 $-(q_D + q_T)h^2$，第二项是 $1 - q_T$；第 231 行是 $q_T R/r$。

第 232 行再计算 $\sqrt{GM/R}\sqrt{B} - R\Omega_{\text{frame}}$。这里同名变量 `vel` 从无量纲括号变为速度，返回值为 $v_\phi = R\Omega$，不是角速度。当前 `Omega0=0`，故不作旋转参考系修正。取本输入后得到 $v_\phi = \sqrt{GM/R}\sqrt{-3h^2 + R/r}$。高度依赖的 R/r 给出了垂直剪切。

10. DiskInnerX1：径向内边界（第 236–281 行）

```
236
237 //----------------------------------------------------------------------------------------
238 //! User-defined boundary Conditions: sets solution in ghost zones to initial values
239
240 void DiskInnerX1(MeshBlock *pmb,Coordinates *pco, AthenaArray<Real> &prim, FaceField &b,
241                  Real time, Real dt,
242                  int il, int iu, int jl, int ju, int kl, int ku, int ngh) {
243   Real rad(0.0), phi(0.0), z(0.0);
244   Real vel;
245   OrbitalVelocityFunc &vK = pmb->porb->OrbitalVelocity;
246   if (std::strcmp(COORDINATE_SYSTEM, "cylindrical") == 0) {
247     for (int k=kl; k<=ku; ++k) {
248       for (int j=jl; j<=ju; ++j) {
249         for (int i=1; i<=ngh; ++i) {
250           GetCylCoord(pco,rad,phi,z,il-i,j,k);
251           prim(IDN,k,j,il-i) = DenProfileCyl(rad,phi,z);
252           vel = VelProfileCyl(rad,phi,z);
253           if (pmb->porb->orbital_advection_defined)
254             vel -= vK(pmb->porb, pco->x1v(il-i), pco->x2v(j), pco->x3v(k));
255           prim(IM1,k,j,il-i) = 0.0;
256           prim(IM2,k,j,il-i) = vel;
257           prim(IM3,k,j,il-i) = 0.0;
258           if (NON_BAROTROPIC_EOS)
259             prim(IEN,k,j,il-i) = PoverR(rad, phi, z)*prim(IDN,k,j,il-i);
260         }
261       }
262     }
263   } else if (std::strcmp(COORDINATE_SYSTEM, "spherical_polar") == 0) {
264     for (int k=kl; k<=ku; ++k) {
265       for (int j=jl; j<=ju; ++j) {
266         for (int i=1; i<=ngh; ++i) {
267           GetCylCoord(pco,rad,phi,z,il-i,j,k);
268           prim(IDN,k,j,il-i) = DenProfileCyl(rad,phi,z);
269           vel = VelProfileCyl(rad,phi,z);
270           if (pmb->porb->orbital_advection_defined)
271             vel -= vK(pmb->porb, pco->x1v(il-i), pco->x2v(j), pco->x3v(k));
272           prim(IM1,k,j,il-i) = 0.0;
273           prim(IM2,k,j,il-i) = 0.0;
274           prim(IM3,k,j,il-i) = vel;
275           if (NON_BAROTROPIC_EOS)
276             prim(IEN,k,j,il-i) = PoverR(rad, phi, z)*prim(IDN,k,j,il-i);
277         }
278       }
279     }
280   }
281 }
```

这个函数填充内侧径向面的幽灵单元。当前输入把这个面设置为 `user`，因此会注册并调用该函数。

函数开头声明柱坐标临时量与旋转速度，并取得可选轨道速度函数。第 246–263 行是柱坐标分支；第 263–280 行是球坐标分支。当前球坐标编译选择后者；两个分支的主要区别是旋转速度存入哪个原始变量分量。

循环让 `i` 从 1 遍历到 `ngh`，定位 `il-i`，即由活动区边缘向外数的每一层幽灵单元。其余两个方向 `k`、`j` 遍历传入的横向索引范围。该索引表示幽灵位置，不是在重新初始化整个活动区域。

每个幽灵单元依次执行：

1. `GetCylCoord()` 取得该幽灵位置的 R, ϕ, z，不会简单复制相邻活动单元的值。
2. `DenProfileCyl()` 按式 (24) 重新计算平衡密度。
3. `VelProfileCyl()` 按式 (25) 计算旋转线速度；若轨道平流启用，再减去对应轨道速度。当前这项附加分支不执行。
4. 柱坐标分支第 255/256/257 行依次设置 $(v_R, v_\phi, v_z) = (0, v_\phi, 0)$。
5. 球坐标分支第 272/273/274 行依次设置 $(v_r, v_\theta, v_\phi) = (0, 0, v_\phi)$。
6. 第 276 行在绝热能量编译下设置 $P = \rho c_s^2$，之后各层循环结束，函数返回。

这里 `prim` 是原始变量数组，速度无需乘密度；`prim(IEN)` 使用第 4 号压强槽位，因为 `IEN=IPR=4`。这与 `u(IEN)` 代表总能量的含义不同。函数没有重新加入随机噪声，也没有使用 `time`、`dt` 改变平衡解。

11. DiskOuterX1：径向外边界（第 282–327 行）

```
282
283 //----------------------------------------------------------------------------------------
284 //! User-defined boundary Conditions: sets solution in ghost zones to initial values
285
286 void DiskOuterX1(MeshBlock *pmb,Coordinates *pco, AthenaArray<Real> &prim, FaceField &b,
287                  Real time, Real dt,
288                  int il, int iu, int jl, int ju, int kl, int ku, int ngh) {
289   Real rad(0.0), phi(0.0), z(0.0);
290   Real vel;
291   OrbitalVelocityFunc &vK = pmb->porb->OrbitalVelocity;
292   if (std::strcmp(COORDINATE_SYSTEM, "cylindrical") == 0) {
293     for (int k=kl; k<=ku; ++k) {
294       for (int j=jl; j<=ju; ++j) {
295         for (int i=1; i<=ngh; ++i) {
296           GetCylCoord(pco,rad,phi,z,iu+i,j,k);
297           prim(IDN,k,j,iu+i) = DenProfileCyl(rad,phi,z);
298           vel = VelProfileCyl(rad,phi,z);
299           if (pmb->porb->orbital_advection_defined)
300             vel -= vK(pmb->porb, pco->x1v(iu+i), pco->x2v(j), pco->x3v(k));
301           prim(IM1,k,j,iu+i) = 0.0;
302           prim(IM2,k,j,iu+i) = vel;
303           prim(IM3,k,j,iu+i) = 0.0;
304           if (NON_BAROTROPIC_EOS)
305             prim(IEN,k,j,iu+i) = PoverR(rad, phi, z)*prim(IDN,k,j,iu+i);
306         }
307       }
308     }
309   } else if (std::strcmp(COORDINATE_SYSTEM, "spherical_polar") == 0) {
310     for (int k=kl; k<=ku; ++k) {
311       for (int j=jl; j<=ju; ++j) {
312         for (int i=1; i<=ngh; ++i) {
313           GetCylCoord(pco,rad,phi,z,iu+i,j,k);
314           prim(IDN,k,j,iu+i) = DenProfileCyl(rad,phi,z);
315           vel = VelProfileCyl(rad,phi,z);
316           if (pmb->porb->orbital_advection_defined)
317             vel -= vK(pmb->porb, pco->x1v(iu+i), pco->x2v(j), pco->x3v(k));
318           prim(IM1,k,j,iu+i) = 0.0;
319           prim(IM2,k,j,iu+i) = 0.0;
320           prim(IM3,k,j,iu+i) = vel;
321           if (NON_BAROTROPIC_EOS)
322             prim(IEN,k,j,iu+i) = PoverR(rad, phi, z)*prim(IDN,k,j,iu+i);
323         }
324       }
325     }
326   }
327 }
```

这个函数填充外侧径向面的幽灵单元。当前输入把这个面设置为 `user`，因此会注册并调用该函数。

函数开头声明柱坐标临时量与旋转速度，并取得可选轨道速度函数。第 292–309 行是柱坐标分支；第 309–326 行是球坐标分支。当前球坐标编译选择后者；两个分支的主要区别是旋转速度存入哪个原始变量分量。

循环让 `i` 从 1 遍历到 `ngh`，定位 `iu+i`，即由活动区边缘向外数的每一层幽灵单元。其余两个方向 `k`、`j` 遍历传入的横向索引范围。该索引表示幽灵位置，不是在重新初始化整个活动区域。

每个幽灵单元依次执行：

1. `GetCylCoord()` 取得该幽灵位置的 R, ϕ, z，不会简单复制相邻活动单元的值。
2. `DenProfileCyl()` 按式 (24) 重新计算平衡密度。

3. `VelProfileCyl()` 按式 (25) 计算旋转线速度；若轨道平流启用，再减去对应轨道速度。当前这项附加分支不执行。
4. 柱坐标分支第 301/302/303 行依次设置 $(v_R, v_\phi, v_z) = (0, v_\phi, 0)$。
5. 球坐标分支第 318/319/320 行依次设置 $(v_r, v_\theta, v_\phi) = (0, 0, v_\phi)$。
6. 第 322 行在绝热能量编译下设置 $P = \rho c_s^2$，之后各层循环结束，函数返回。

这里 `prim` 是原始变量数组，速度无需乘密度；`prim(IEN)` 使用第 4 号压强槽位，因为 `IEN=IPR=4`。这与 `u(IEN)` 代表总能量的含义不同。函数没有重新加入随机噪声，也没有使用 `time`、`dt` 改变平衡解。

12. DiskInnerX2：极角内边界（第 328–373 行）

```
328
329 //----------------------------------------------------------------------------------------
330 //! User-defined boundary Conditions: sets solution in ghost zones to initial values
331
332 void DiskInnerX2(MeshBlock *pmb,Coordinates *pco, AthenaArray<Real> &prim, FaceField &b,
333                  Real time, Real dt,
334                  int il, int iu, int jl, int ju, int kl, int ku, int ngh) {
335   Real rad(0.0), phi(0.0), z(0.0);
336   Real vel;
337   OrbitalVelocityFunc &vK = pmb->porb->OrbitalVelocity;
338   if (std::strcmp(COORDINATE_SYSTEM, "cylindrical") == 0) {
339     for (int k=kl; k<=ku; ++k) {
340       for (int j=1; j<=ngh; ++j) {
341         for (int i=il; i<=iu; ++i) {
342           GetCylCoord(pco,rad,phi,z,i,jl-j,k);
343           prim(IDN,k,jl-j,i) = DenProfileCyl(rad,phi,z);
344           vel = VelProfileCyl(rad,phi,z);
345           if (pmb->porb->orbital_advection_defined)
346             vel -= vK(pmb->porb, pco->x1v(i), pco->x2v(jl-j), pco->x3v(k));
347           prim(IM1,k,jl-j,i) = 0.0;
348           prim(IM2,k,jl-j,i) = vel;
349           prim(IM3,k,jl-j,i) = 0.0;
350           if (NON_BAROTROPIC_EOS)
351             prim(IEN,k,jl-j,i) = PoverR(rad, phi, z)*prim(IDN,k,jl-j,i);
352         }
353       }
354     }
355   } else if (std::strcmp(COORDINATE_SYSTEM, "spherical_polar") == 0) {
356     for (int k=kl; k<=ku; ++k) {
357       for (int j=1; j<=ngh; ++j) {
358         for (int i=il; i<=iu; ++i) {
359           GetCylCoord(pco,rad,phi,z,i,jl-j,k);
360           prim(IDN,k,jl-j,i) = DenProfileCyl(rad,phi,z);
361           vel = VelProfileCyl(rad,phi,z);
362           if (pmb->porb->orbital_advection_defined)
363             vel -= vK(pmb->porb, pco->x1v(i), pco->x2v(jl-j), pco->x3v(k));
364           prim(IM1,k,jl-j,i) = 0.0;
365           prim(IM2,k,jl-j,i) = 0.0;
366           prim(IM3,k,jl-j,i) = vel;
367           if (NON_BAROTROPIC_EOS)
368             prim(IEN,k,jl-j,i) = PoverR(rad, phi, z)*prim(IDN,k,jl-j,i);
369         }
370       }
371     }
372   }
373 }
```

这个函数填充内侧极角面的幽灵单元。当前输入把这个面设置为 `user`，因此会注册并调用该函数。

函数开头声明柱坐标临时量与旋转速度，并取得可选轨道速度函数。第 338–355 行是柱坐标分支；第 355–372 行是球坐标分支。当前球坐标编译选择后者；两个分支的主要区别是旋转速度存入哪个原始变量分量。

循环让 `j` 从 1 遍历到 `ngh`，定位 `jl-j`，即由活动区边缘向外数的每一层幽灵单元。其余两个方向 `k`、`i` 遍历传入的横向索引范围。该索引表示幽灵位置，不是在重新初始化整个活动区域。

每个幽灵单元依次执行：

1. `GetCylCoord()` 取得该幽灵位置的 R, ϕ, z，不会简单复制相邻活动单元的值。
2. `DenProfileCyl()` 按式 (24) 重新计算平衡密度。
3. `VelProfileCyl()` 按式 (25) 计算旋转线速度；若轨道平流启用，再减去对应轨道速度。当前这项附加分支不执行。

4. 柱坐标分支第 347/348/349 行依次设置 $(v_R, v_\phi, v_z) = (0, v_\phi, 0)$。
5. 球坐标分支第 364/365/366 行依次设置 $(v_r, v_\theta, v_\phi) = (0, 0, v_\phi)$。
6. 第 368 行在绝热能量编译下设置 $P = \rho c_s^2$，之后各层循环结束，函数返回。

这里 `prim` 是原始变量数组，速度无需乘密度；`prim(IEN)` 使用第 4 号压强槽位，因为 `IEN=IPR=4`。这与 `u(IEN)` 代表总能量的含义不同。函数没有重新加入随机噪声，也没有使用 `time`、`dt` 改变平衡解。

13. DiskOuterX2：极角外边界（第 374–419 行）

```
374
375 //----------------------------------------------------------------------------------------
376 //! User-defined boundary Conditions: sets solution in ghost zones to initial values
377
378 void DiskOuterX2(MeshBlock *pmb,Coordinates *pco, AthenaArray<Real> &prim, FaceField &b,
379                  Real time, Real dt,
380                  int il, int iu, int jl, int ju, int kl, int ku, int ngh) {
381   Real rad(0.0), phi(0.0), z(0.0);
382   Real vel;
383   OrbitalVelocityFunc &vK = pmb->porb->OrbitalVelocity;
384   if (std::strcmp(COORDINATE_SYSTEM, "cylindrical") == 0) {
385     for (int k=kl; k<=ku; ++k) {
386       for (int j=1; j<=ngh; ++j) {
387         for (int i=il; i<=iu; ++i) {
388           GetCylCoord(pco,rad,phi,z,i,ju+j,k);
389           prim(IDN,k,ju+j,i) = DenProfileCyl(rad,phi,z);
390           vel = VelProfileCyl(rad,phi,z);
391           if (pmb->porb->orbital_advection_defined)
392             vel -= vK(pmb->porb, pco->x1v(i), pco->x2v(ju+j), pco->x3v(k));
393           prim(IM1,k,ju+j,i) = 0.0;
394           prim(IM2,k,ju+j,i) = vel;
395           prim(IM3,k,ju+j,i) = 0.0;
396           if (NON_BAROTROPIC_EOS)
397             prim(IEN,k,ju+j,i) = PoverR(rad, phi, z)*prim(IDN,k,ju+j,i);
398         }
399       }
400     }
401   } else if (std::strcmp(COORDINATE_SYSTEM, "spherical_polar") == 0) {
402     for (int k=kl; k<=ku; ++k) {
403       for (int j=1; j<=ngh; ++j) {
404         for (int i=il; i<=iu; ++i) {
405           GetCylCoord(pco,rad,phi,z,i,ju+j,k);
406           prim(IDN,k,ju+j,i) = DenProfileCyl(rad,phi,z);
407           vel = VelProfileCyl(rad,phi,z);
408           if (pmb->porb->orbital_advection_defined)
409             vel -= vK(pmb->porb, pco->x1v(i), pco->x2v(ju+j), pco->x3v(k));
410           prim(IM1,k,ju+j,i) = 0.0;
411           prim(IM2,k,ju+j,i) = 0.0;
412           prim(IM3,k,ju+j,i) = vel;
413           if (NON_BAROTROPIC_EOS)
414             prim(IEN,k,ju+j,i) = PoverR(rad, phi, z)*prim(IDN,k,ju+j,i);
415         }
416       }
417     }
418   }
419 }
```

这个函数填充外侧极角面的幽灵单元。当前输入把这个面设置为 `user`，因此会注册并调用该函数。

函数开头声明柱坐标临时量与旋转速度，并取得可选轨道速度函数。第 384–401 行是柱坐标分支；第 401–418 行是球坐标分支。当前球坐标编译选择后者；两个分支的主要区别是旋转速度存入哪个原始变量分量。

循环让 `j` 从 1 遍历到 `ngh`，定位 `ju+j`，即由活动区边缘向外数的每一层幽灵单元。其余两个方向 `k`、`i` 遍历传入的横向索引范围。该索引表示幽灵位置，不是在重新初始化整个活动区域。

每个幽灵单元依次执行：

1. `GetCylCoord()` 取得该幽灵位置的 R, ϕ, z，不会简单复制相邻活动单元的值。
2. `DenProfileCyl()` 按式 (24) 重新计算平衡密度。
3. `VelProfileCyl()` 按式 (25) 计算旋转线速度；若轨道平流启用，再减去对应轨道速度。当前这项附加分支不执行。
4. 柱坐标分支第 393/394/395 行依次设置 $(v_R, v_\phi, v_z) = (0, v_\phi, 0)$。

5. 球坐标分支第 410/411/412 行依次设置 $(v_r, v_\theta, v_\phi) = (0, 0, v_\phi)$。
6. 第 414 行在绝热能量编译下设置 $P = \rho c_s^2$，之后各层循环结束，函数返回。

这里 `prim` 是原始变量数组，速度无需乘密度；`prim(IEN)` 使用第 4 号压强槽位，因为 `IEN=IPR=4`。这与 `u(IEN)` 代表总能量的含义不同。函数没有重新加入随机噪声，也没有使用 `time`、`dt` 改变平衡解。

14. DiskInnerX3：方位角内边界（第 420–465 行）

```
420
421 //----------------------------------------------------------------------------------------
422 //! User-defined boundary Conditions: sets solution in ghost zones to initial values
423
424 void DiskInnerX3(MeshBlock *pmb,Coordinates *pco, AthenaArray<Real> &prim, FaceField &b,
425                  Real time, Real dt,
426                  int il, int iu, int jl, int ju, int kl, int ku, int ngh) {
427   Real rad(0.0), phi(0.0), z(0.0);
428   Real vel;
429   OrbitalVelocityFunc &vK = pmb->porb->OrbitalVelocity;
430   if (std::strcmp(COORDINATE_SYSTEM, "cylindrical") == 0) {
431     for (int k=1; k<=ngh; ++k) {
432       for (int j=jl; j<=ju; ++j) {
433         for (int i=il; i<=iu; ++i) {
434           GetCylCoord(pco,rad,phi,z,i,j,kl-k);
435           prim(IDN,kl-k,j,i) = DenProfileCyl(rad,phi,z);
436           vel = VelProfileCyl(rad,phi,z);
437           if (pmb->porb->orbital_advection_defined)
438             vel -= vK(pmb->porb, pco->x1v(i), pco->x2v(j), pco->x3v(kl-k));
439           prim(IM1,kl-k,j,i) = 0.0;
440           prim(IM2,kl-k,j,i) = vel;
441           prim(IM3,kl-k,j,i) = 0.0;
442           if (NON_BAROTROPIC_EOS)
443             prim(IEN,kl-k,j,i) = PoverR(rad, phi, z)*prim(IDN,kl-k,j,i);
444         }
445       }
446     }
447   } else if (std::strcmp(COORDINATE_SYSTEM, "spherical_polar") == 0) {
448     for (int k=1; k<=ngh; ++k) {
449       for (int j=jl; j<=ju; ++j) {
450         for (int i=il; i<=iu; ++i) {
451           GetCylCoord(pco,rad,phi,z,i,j,kl-k);
452           prim(IDN,kl-k,j,i) = DenProfileCyl(rad,phi,z);
453           vel = VelProfileCyl(rad,phi,z);
454           if (pmb->porb->orbital_advection_defined)
455             vel -= vK(pmb->porb, pco->x1v(i), pco->x2v(j), pco->x3v(kl-k));
456           prim(IM1,kl-k,j,i) = 0.0;
457           prim(IM2,kl-k,j,i) = 0.0;
458           prim(IM3,kl-k,j,i) = vel;
459           if (NON_BAROTROPIC_EOS)
460             prim(IEN,kl-k,j,i) = PoverR(rad, phi, z)*prim(IDN,kl-k,j,i);
461         }
462       }
463     }
464   }
465 }
```

这个函数填充内侧方位角面的幽灵单元。当前输入将 X3 设置为 `periodic`，所以这个用户边界函数不会被注册或调用。函数仍保留在文件中，只有把对应面显式设为 `user` 才使用。

函数开头声明柱坐标临时量与旋转速度，并取得可选轨道速度函数。第 430–447 行是柱坐标分支；第 447–464 行是球坐标分支。当前球坐标编译选择后者；两个分支的主要区别是旋转速度存入哪个原始变量分量。

循环让 `k` 从 1 遍历到 `ngh`，定位 `kl-k`，即由活动区边缘向外数的每一层幽灵单元。其余两个方向 `j`、`i` 遍历传入的横向索引范围。该索引表示幽灵位置，不是在重新初始化整个活动区域。

每个幽灵单元依次执行：

1. `GetCylCoord()` 取得该幽灵位置的 R, ϕ, z，不会简单复制相邻活动单元的值。
2. `DenProfileCyl()` 按式 (24) 重新计算平衡密度。
3. `VelProfileCyl()` 按式 (25) 计算旋转线速度；若轨道平流启用，再减去对应轨道速度。当前这项附加分支不执行。
4. 柱坐标分支第 439/440/441 行依次设置 $(v_R, v_\phi, v_z) = (0, v_\phi, 0)$。
5. 球坐标分支第 456/457/458 行依次设置 $(v_r, v_\theta, v_\phi) = (0, 0, v_\phi)$。

6. 第 460 行在绝热能量编译下设置 $P = \rho c_s^2$，之后各层循环结束，函数返回。

这里 `prim` 是原始变量数组，速度无需乘密度；`prim(IEN)` 使用第 4 号压强槽位，因为 `IEN=IPR=4`。这与 `u(IEN)` 代表总能量的含义不同。函数没有重新加入随机噪声，也没有使用 `time`、`dt` 改变平衡解。

15. DiskOuterX3：方位角外边界（第 466–511 行）

```
466
467 //----------------------------------------------------------------------------------------
468 //! User-defined boundary Conditions: sets solution in ghost zones to initial values
469
470 void DiskOuterX3(MeshBlock *pmb,Coordinates *pco, AthenaArray<Real> &prim, FaceField &b,
471                  Real time, Real dt,
472                  int il, int iu, int jl, int ju, int kl, int ku, int ngh) {
473   Real rad(0.0), phi(0.0), z(0.0);
474   Real vel;
475   OrbitalVelocityFunc &vK = pmb->porb->OrbitalVelocity;
476   if (std::strcmp(COORDINATE_SYSTEM, "cylindrical") == 0) {
477     for (int k=1; k<=ngh; ++k) {
478       for (int j=jl; j<=ju; ++j) {
479         for (int i=il; i<=iu; ++i) {
480           GetCylCoord(pco,rad,phi,z,i,j,ku+k);
481           prim(IDN,ku+k,j,i) = DenProfileCyl(rad,phi,z);
482           vel = VelProfileCyl(rad,phi,z);
483           if (pmb->porb->orbital_advection_defined)
484             vel -= vK(pmb->porb, pco->x1v(i), pco->x2v(j), pco->x3v(ku+k));
485           prim(IM1,ku+k,j,i) = 0.0;
486           prim(IM2,ku+k,j,i) = vel;
487           prim(IM3,ku+k,j,i) = 0.0;
488           if (NON_BAROTROPIC_EOS)
489             prim(IEN,ku+k,j,i) = PoverR(rad, phi, z)*prim(IDN,ku+k,j,i);
490         }
491       }
492     }
493   } else if (std::strcmp(COORDINATE_SYSTEM, "spherical_polar") == 0) {
494     for (int k=1; k<=ngh; ++k) {
495       for (int j=jl; j<=ju; ++j) {
496         for (int i=il; i<=iu; ++i) {
497           GetCylCoord(pco,rad,phi,z,i,j,ku+k);
498           prim(IDN,ku+k,j,i) = DenProfileCyl(rad,phi,z);
499           vel = VelProfileCyl(rad,phi,z);
500           if (pmb->porb->orbital_advection_defined)
501             vel -= vK(pmb->porb, pco->x1v(i), pco->x2v(j), pco->x3v(ku+k));
502           prim(IM1,ku+k,j,i) = 0.0;
503           prim(IM2,ku+k,j,i) = 0.0;
504           prim(IM3,ku+k,j,i) = vel;
505           if (NON_BAROTROPIC_EOS)
506             prim(IEN,ku+k,j,i) = PoverR(rad, phi, z)*prim(IDN,ku+k,j,i);
507         }
508       }
509     }
510   }
511 }
```

这个函数填充外侧方位角面的幽灵单元。当前输入将 X3 设置为 `periodic`，所以这个用户边界函数不会被注册或调用。函数仍保留在文件中，只有把对应面显式设为 `user` 才使用。

函数开头声明柱坐标临时量与旋转速度，并取得可选轨道速度函数。第 476–493 行是柱坐标分支；第 493–510 行是球坐标分支。当前球坐标编译选择后者；两个分支的主要区别是旋转速度存入哪个原始变量分量。

循环让 `k` 从 1 遍历到 `ngh`，定位 `ku+k`，即由活动区边缘向外数的每一层幽灵单元。其余两个方向 `j`、`i` 遍历传入的横向索引范围。该索引表示幽灵位置，不是在重新初始化整个活动区域。

每个幽灵单元依次执行：

1. `GetCylCoord()` 取得该幽灵位置的 R, ϕ, z，不会简单复制相邻活动单元的值。
2. `DenProfileCyl()` 按式 (24) 重新计算平衡密度。
3. `VelProfileCyl()` 按式 (25) 计算旋转线速度；若轨道平流启用，再减去对应轨道速度。当前这项附加分支不执行。
4. 柱坐标分支第 485/486/487 行依次设置 $(v_R, v_\phi, v_z) = (0, v_\phi, 0)$。
5. 球坐标分支第 502/503/504 行依次设置 $(v_r, v_\theta, v_\phi) = (0, 0, v_\phi)$。
6. 第 506 行在绝热能量编译下设置 $P = \rho c_s^2$，之后各层循环结束，函数返回。

这里 `prim` 是原始变量数组，速度无需乘密度；`prim(IEN)` 使用第 4 号压强槽位，因为 `IEN=IPR=4`。这与 `u(IEN)` 代表总能量的含义不同。函数没有重新加入随机噪声，也没有使用 `time`、`dt` 改变平衡解。

16. UserWorkInLoop：式 (27) 的逐步热松弛（第 512–528 行）

```
512
513 // Eq. (27), exact exponential relaxation once per completed timestep.
514 // Update both U and W because Athena computes the next timestep from W.
515 void MeshBlock::UserWorkInLoop() {
516   const Real f=-std::expm1(-pmy_mesh->dt/tau);
517   for(int k=ks;k<=ke;++k) for(int j=js;j<=je;++j) for(int i=is;i<=ie;++i) {
518     const Real rho=phydro->u(IDN,k,j,i);
519     const Real kinetic=0.5*(SQR(phydro->u(IM1,k,j,i))+
520       SQR(phydro->u(IM2,k,j,i))+SQR(phydro->u(IM3,k,j,i)))/rho;
521     Real R,phi,z; GetCylCoord(pcoord,R,phi,z,i,j,k);
522     const Real target=rho*PoverR(R,phi,z)/(gamma_gas-1.0);
523     const Real internal=phydro->u(IEN,k,j,i)-kinetic;
524     const Real cooled=internal+f*(target-internal);
525     phydro->u(IEN,k,j,i)=kinetic+cooled;
526     phydro->w(IPR,k,j,i)=(gamma_gas-1.0)*cooled;
527   }
528 }
```

该函数在每个完整流体时间步之后执行。第 513–514 行说明作用：实施冷却，同时保持两类热变量一致。

第 516 行计算 $f = 1 - \exp(-\Delta t/\tau)$。`expm1(x)` 返回 $e^x - 1$，所以前面的负号得到论文式 (27) 的正松弛系数。`pmy_mesh->dt` 是当前完整步长，不是代码自行设定的额外时间。

第 517 行遍历活动单元；第 518 行从守恒数组读取密度。第 519–520 行使用动量密度计算动能密度：

$$K = \frac{m_r^2 + m_\theta^2 + m_\phi^2}{2\rho}.$$

第 521 行转换单元坐标。第 522 行用当前密度计算目标内能密度：

$$\epsilon_0 = \frac{\rho c_s^2(R)}{\gamma - 1}.$$

第 523 行由 $E - K$ 取得当前内能。第 524 行进行热松弛：

$$\epsilon_{\rm new} = \epsilon + (\epsilon_0 - \epsilon)[1 - \exp(-\Delta t/\tau)].$$

在这个子步骤中 ρ 不变且 $\epsilon \propto \rho T$，因此等价于论文对温度的更新。第 525 行把原动能加回总能量，第 526 行更新原始压强 $P = (\gamma - 1)\epsilon_{\rm new}$；第 527–528 行结束循环与函数。

这个函数不改变密度或动量，也不会把总能量直接覆盖成内能。对于当前极小的 `tau`，通常 f 在数值上接近 1，温度迅速回到局部目标。同步更新 `w(IPR)` 是为了后续读取原始变量的时间步估计使用更新后的压强。

二、inputs/athinput.figure2

下面按输入文件的原顺序解释每一项。尖括号包围的块标题（例如 `<mesh>`）开始一个参数块，直到下一个块标题；同名参数在不同块中的含义可以不同，例如 `<mesh>` 的 `nx1` 是总网格数，`<meshblock>` 的 `nx1` 是单块网格数。下面所有时间值均为代码单位，$GM = R_0 = 1$ 时 $P_0 = 2\pi$。

1. comment 和 job（第 1–8 行）

```
<comment>
problem = Svanberg et al. 2022, Section 3, Figure 2


<job>
problem_id = vsi


```

- 第 1 行 `<comment>`：开始描述块。
- 第 2 行 `problem=...`：问题说明文字，不参与动力学计算，也不会自动载入论文参数。
- 第 5 行 `<job>`：开始任务命名块。
- 第 6 行 `problem_id=vsi`：输出文件的基础名称为 `vsi`，不改变物理初态。

2. time：步长控制与终止时间（第 9–17 行）

```
<time>
cfl_number = 0.3
nlim = -1
tlim = 3141.592653589793
integrator = vl2
xorder = 2
ncycle_out = 100


```

- 第 9 行 `<time>`：开始时间控制块。
- 第 10 行 `cfl_number=0.3`：控制流体 CFL 步长的安全系数，不表示固定时间步长为 0.3。
- 第 11 行 `nlim=-1`：不设置最大步数限制。
- 第 12 行 `tlim=3141.592653589793`：$500 \times 2\pi = 500P_0$，即 Figure 2 的目标时刻。论文整段 $3000P_0$ 对应 18849.55592153876，不是这里当前的值。
- 第 13 行 `integrator=vl2`：采用两阶段、二阶时间精度的 van Leer 时间积分。
- 第 14 行 `xorder=2`：采用二阶空间重构，在这里对应 PLM。
- 第 15 行 `ncycle_out=100`：每 100 个流体步在标准输出打印一次循环信息；不是每 100 步保存一次完整快照。

3. mesh：整个计算域（第 18–37 行）

```
<mesh>
nx1 = 704
x1min = 1.0
x1max = 30.0
x1rat = 1.005
ix1_bc = user
ox1_bc = user
nx2 = 448
x2min = 1.3207963267948966
x2max = 1.8207963267948966
ix2_bc = user
ox2_bc = user
nx3 = 1
x3min = 0.0
x3max = 6.283185307179586
ix3_bc = periodic
ox3_bc = periodic
num_threads = 4


```

- 第 18 行 `<mesh>`：整个网格的参数块。
- 第 19 行 `nx1=704`：径向活动单元总数为 704。

- 第 20 行 `x1min=1.0`：球半径内边界 $r = 1$。
- 第 21 行 `x1max=30.0`：球半径外边界 $r = 30$。
- 第 22 行 `x1rat=1.005`：相邻径向单元宽度之比为 1.005；端点保持为 1 和 30。对应单元面可写为 $r_i = 1 + 29(q^i - 1)/(q^{704} - 1)$，$q = 1.005$。
- 第 23 行 `ix1_bc=user`：径向内边界调用 `DiskInnerX1()`。
- 第 24 行 `ox1_bc=user`：径向外边界调用 `DiskOuterX1()`。
- 第 25 行 `nx2=448`：极角方向活动单元总数为 448。
- 第 26 行 `x2min=1.3207963267948966`：极角内边界 $\pi/2 - 5h$，其中 $h = 0.05$。
- 第 27 行 `x2max=1.8207963267948966`：极角外边界 $\pi/2 + 5h$。这些角度使用弧度。
- 第 28 行 `ix2_bc=user`：极角内边界调用 `DiskInnerX2()`。
- 第 29 行 `ox2_bc=user`：极角外边界调用 `DiskOuterX2()`。
- 第 30 行 `nx3=1`：只有一个方位角单元，不解析方位角变化；仍然演化方位速度与动量。
- 第 31 行 `x3min=0.0`：方位角坐标起点为 0。
- 第 32 行 `x3max=6.283185307179586`：方位角坐标终点为 2π。
- 第 33 行 `ix3_bc=periodic`：方位角内边界为周期条件，不调用 `DiskInnerX3()`。
- 第 34 行 `ox3_bc=periodic`：方位角外边界为周期条件，不调用 `DiskOuterX3()`。
- 第 35 行 `num_threads=4`：设置网格处理使用的 OpenMP 线程数为 4；需配合启用 OpenMP 的可执行程序。

输入未写 `x2rat`，因此采用默认值 1，极角均匀划分。极角跨度 $0.5 = 10h$，448 格在中面附近对应约 44.8 格/H，与论文文字中的约 12 格/H 不一致；这里实际执行的是明确列出的范围和 448 格。也没有另外设置“10 格/H”的参数来覆盖径向网格。

4. meshblock：单个网格块（第 38–43 行）

```
<meshblock>
nx1 = 176
nx2 = 448
nx3 = 1


```

- 第 38 行 `<meshblock>`：开始单块网格配置。
- 第 39 行 `nx1=176`：每块有 176 个径向活动单元。
- 第 40 行 `nx2=448`：每块覆盖全部 448 个极角活动单元。
- 第 41 行 `nx3=1`：每块只有一个方位角活动单元。

总块数为 $(704/176)(448/448)(1/1) = 4$。网格分块用于计算组织，没有改变总分辨率；幽灵单元不计入上述活动单元数。

5. hydro：状态方程与底限（第 44–49 行）

```
<hydro>
gamma = 1.4
dfloor = 1.e-12
pfloor = 1.e-16


```

- 第 44 行 `<hydro>`：开始流体参数块。
- 第 45 行 `gamma=1.4`：$\gamma = 7/5$，用于 $P = (\gamma - 1)\epsilon$。`InitUserMeshData()` 读取为 `gamma_gas`，初始化能量和冷却都使用它。
- 第 46 行 `dfloor=1.e-12`：密度数值底限。`DenProfileCyl()` 返回值也通过该底限截断。
- 第 47 行 `pfloor=1.e-16`：压强数值底限，由流体状态处理使用；本文件没有把它读成另一个独立全局变量。

底限用于数值保护，不是额外的密度或温度幂律。

6. problem：盘模型、噪声与冷却（第 50–61 行）

```
<problem>
GM = 1.0
r0 = 1.0
rho0 = 1.0
p0_over_r0 = 0.0025
pslope = -1.0
```

```
dslope = -2.0
noise_amp = 0.01
seed = 202211
tau = 6.283185307179586e-10


```

- 第 50 行 `<problem>`：开始问题生成器参数块。
- 第 51 行 `GM=1.0`：中心引力参数。既用于本文件的解析平衡，也启用内置点质量引力；本文件没有再手写一次重复引力。
- 第 52 行 `r0=1.0`：柱半径参考位置 $R_0 = 1$。
- 第 53 行 `rho0=1.0`：$R = R_0$ 的中面密度归一化为 1。
- 第 54 行 `p0_over_r0=0.0025`：目标 P/ρ 在参考半径的值，即 c_0^2；对应 $c_0 = 0.05$，不是 $c_0 = 0.0025$。
- 第 55 行 `pslope=-1.0`：$c_s^2 \propto R^{-1}$，对应论文 $q_T = 1$。
- 第 56 行 `dslope=-2.0`：中面密度 $\rho_{\rm mid} \propto R^{-2}$，对应论文 $q_D = 2$。
- 第 57 行 `noise_amp=0.01`：每个速度分量噪声的最大绝对幅度为局部声速的 1%，由第 170–172 行乘入动量增量。
- 第 58 行 `seed=202211`：散列噪声的种子，决定可重复的初始扰动。
- 第 59 行 `tau=6.283185307179586e-10`：$10^{-10} P_0$ 换算后的代码冷却时间，不是 10^{-10} 代码时间。

这些参数给出 $h^2 = c_s^2/(GM/R) = 0.0025$，故 $h = 0.05$。输入没有单独的 `h` 字段；它已由声速、引力与径向幂律组合确定。

7. output1：完整流场快照（第 62–67 行）

```
<output1>
file_type = vtk
variable = prim
dt = 6.283185307179586


```

- 第 62 行 `<output1>`：第一组输出配置。
- 第 63 行 `file_type=vtk`：以 VTK 格式保存空间场。
- 第 64 行 `variable=prim`：保存原始流体变量，包括密度、压强与速度；不是只保存二维图片。
- 第 65 行 `dt=6.283185307179586`：输出间隔 $2\pi = 1 P_0$。

这里的 `dt` 是该输出块的采样间隔，不是 `<time>` 控制的流体积分步长。常规输出在首个达到或跨过计划时刻的步写出，所以准确输出时间需读取文件头。

8. output2：检查点（第 68–72 行）

```
<output2>
file_type = rst
dt = 62.83185307179586


```

- 第 68 行 `<output2>`：第二组输出配置。
- 第 69 行 `file_type=rst`：保存可供重启的检查点，而非可视化快照。
- 第 70 行 `dt=62.83185307179586`：间隔 $20\pi = 10 P_0$。

这与 VTK 的用途不同：检查点保存继续积分需要的状态。

9. output3：history（第 73–75 行）

```
<output3>
file_type = hst
dt = 0.6283185307179586
```

- 第 73 行 `<output3>`：第三组输出配置。
- 第 74 行 `file_type=hst`：保存 history 时间序列，例如积分量。
- 第 75 行 `dt=0.6283185307179586`：间隔 $0.2\pi = 0.1 P_0$。

history 不保存每个空间单元的完整流场，不能代替 VTK 快照。